

Security-Auditing Controls

Structured, Append-Only Audit Records with SIEM Detection

A plain-English explanation of the design, every component, the end-to-end workflow and the architecture

Item	Detail
What this document is	A complete, non-technical-friendly explanation of a security-auditing capability: what it records, how each part works, how a record travels end to end, and how alerts are produced.
Scope of the control	Authentication events, authorization decisions, certificate (PKI) activity and privileged operations.
Record format	Structured, append-only records carrying at minimum actor, resource, action and trace ID.
Detection scope	SIEM alerting on repeated authentication failures and on abnormal access behaviour.
Intended audience	Engineers, security analysts, auditors, risk and compliance teams, and managers who need the plain-English version.
How to read it	Section 1 is the summary. Sections 4–6 explain every component. Section 7 walks the workflow. Section 9 contains the architecture diagram and its explanation.

Contents

1. Executive Summary — the whole thing in plain English
2. The Problem This Solves
3. Key Ideas Explained Simply
4. What Exactly Gets Recorded — the four event families
5. The Audit Record — field by field
6. Components, End to End
 - 6.1 Layer 1 — Event Sources
 - 6.2 Layer 2 — Capture and Shaping
 - 6.3 Layer 3 — Transport
 - 6.4 Layer 4 — The Append-Only Audit Store
 - 6.5 Layer 5 — SIEM and Detection
 - 6.6 Layer 6 — Alert, Response and Evidence
 - 6.7 Layer 7 — Cross-Cutting Controls
7. The Workflow, End to End
8. Detection Logic in Detail
9. Architecture Diagram and Its Explanation
10. Design Rules and Non-Functional Requirements
11. Failure Modes and Safeguards
12. Glossary

1. Executive Summary — the whole thing in plain English

This project builds a **security black box** for our systems: a trustworthy written record of every security-significant thing that happens, plus an automatic watchdog that reads that record and raises the alarm when something looks wrong.

Think of a bank vault. Four things are worth writing down: who proved their identity at the door, what each person was allowed to open once inside, which keys were cut and handed out, and what the managers did with their special privileges. Our systems have exactly the same four things, and this project makes sure all four are written down properly.

The four things we record

- **Authentication events** — every attempt to prove identity — logins, multi-factor checks, token issuance, and every failure.
- **Authorization decisions** — every time the system decided whether an actor was allowed to do something — the allows as well as the denials.
- **Certificate activity** — every digital certificate issued, renewed, revoked or expired, and every failed certificate handshake.
- **Privileged operations** — every administrative or high-power action — configuration changes, secret access, role grants, bulk data exports.

How we record them

Every record is **structured** — it is a set of labelled fields rather than a sentence, so a computer can count and compare records instead of reading English. Every record answers four questions in fixed fields: **actor** (who did it), **resource** (what they did it to), **action** (what they did), and **trace ID** (a shared reference number that ties together everything belonging to the same original request).

Every record is also **append-only**: once written it can be read but never edited or deleted, and each record is cryptographically linked to the one before it. That is what turns a log into evidence. A log an attacker can edit proves nothing.

What we do with them

The records flow continuously into a SIEM — a security analytics platform. The SIEM runs two families of detection. The first counts repeated authentication failures, which is how password guessing, spraying and credential-stuffing attacks reveal themselves. The second learns what normal access looks like for each person and each service, and flags what does not fit: a login from a new country, an admin action at 3 a.m., a dormant account waking up, a sudden bulk export. When something fires, an alert goes to the on-call responder with the trace ID attached, so the whole story can be replayed in one search.

In one sentence: we write down every security-relevant action in a fixed, tamper-evident format, tie related actions together with a shared reference number, and have a machine watch the stream so that failures and unusual behaviour surface in minutes rather than months.

2. The Problem This Solves

Before a control like this exists, most organisations are in one of these situations, and usually all of them at once:

- **Logs exist but say different things.** Each team invented its own format and wording, so the same event looks different in five places and nothing can be counted reliably.
- **Logs are prose, not data.** A line like “user bob could not sign in” cannot be counted, grouped or graphed. A structured record can.
- **Related events cannot be joined.** The login is in one system, the permission check in another and the admin action in a third, with nothing linking them. Investigators join them by guessing from timestamps.
- **Denials are thrown away.** Many systems only log success. Denials are where attacks are most visible.
- **Logs can be changed.** If an administrator (or an intruder who has become one) can edit or delete log lines, the log cannot be used as evidence.
- **Nobody is watching.** Even good logs sitting in storage do not stop anything. Detection has to be automatic and continuous.
- **Failure is silent.** When a log source quietly stops sending, dashboards go calm and everyone assumes a quiet day.

This project addresses all seven: one shared format, machine-readable fields, a shared trace ID, denials recorded at the same fidelity as allows, storage that physically cannot be edited, continuous automated detection, and monitoring of the pipeline itself so that silence raises an alarm.

Who benefits, and how

Group	What they get from it
Security analysts	A single searchable trail. One trace ID replays a whole request; one actor query shows everything an identity did that day.
Engineers and service owners	Fast answers to “who changed this and when”, plus early warning on certificates about to expire.
Auditors and compliance	Provable evidence rather than assertions: complete records, immutable storage, demonstrable retention and access control.
Incident responders	Minutes-to-answer instead of days, and pre-approved containment actions that can run automatically.
Management and risk	Measurable coverage: which systems emit audit records, how many privileged actions occur, how quickly alerts are handled.

3. Key Ideas Explained Simply

Six ideas carry most of the design. Everything else is plumbing around them.

3.1 A structured record

A structured record is a form with labelled boxes rather than a sentence. Instead of writing “admin jane granted bob the payments-admin role on 1 September”, the system writes a small set of fields: actor = jane, action = grant_role, resource = payments-admin, target = bob, outcome = success, time = ..., trace ID = A person can still read it, but crucially a computer can now count it, group it, compare it against yesterday and alert on it. The wording of a sentence changes every time a developer edits the code; the field names do not.

3.2 Actor, resource, action — and outcome

These are the “who, what to, what” of every record, and they are mandatory.

- **Actor:** the identity that performed the action. It can be a person, a service account, or an automated system. We store a stable identifier (which does not change when someone changes their name) as well as a readable display name.
- **Resource:** the thing acted upon — an account, a document, a database, a configuration item, a certificate. We store its type, its identifier, its owner and its sensitivity classification.
- **Action:** what was attempted, taken from a fixed list of allowed values such as authenticate, authorize, read, export, grant_role, issue_certificate, revoke_certificate.
- **Outcome:** how it ended — success, failure or deny — with a reason code from a fixed list, such as bad_password, mfa_denied, policy_denied or certificate_expired.

3.3 The trace ID

A trace ID is a reference number generated the moment a request first arrives, then carried unchanged through every service that touches that request. Every audit record produced along the way is stamped with it.

This is the single most useful field in the whole design. A user reports a problem, or an alert fires; one search on the trace ID returns the login, the permission check, the certificate used for the internal call and the privileged action at the end, in order, across four different systems. Without it, an analyst reconstructs the story by hand from timestamps and hopes nothing overlapped.

3.4 Append-only

Append-only means new records can be added and existing records can be read, but nothing can be modified or removed. This is enforced by the storage platform itself — permissions that grant no delete rights to any identity, and immutability settings applied at the moment of writing — not by a policy document asking people not to.

Additionally each record carries a cryptographic hash of the record before it, forming a chain. Altering any single record changes its hash and therefore breaks every link after it, which a scheduled verifier will notice. Blocks of

the chain are signed and the signature is published to a separate system, so no single administrator controls both the data and the proof.

3.5 What a SIEM is

A SIEM (Security Information and Event Management platform) is the security team's search engine and watchtower. It continuously reads the stream of audit records, indexes every field for fast search, and runs detection rules against them as they arrive. It produces alerts, dashboards and investigation tooling. Importantly, the SIEM holds a working copy — the append-only store remains the authoritative record.

3.6 The two detection ideas

Repeated authentication failures is counting. Real users mistype a password once or twice; attackers generate tens, hundreds or thousands of failures, or a handful of failures against very many accounts. Counting failures across different dimensions — per account, per source address, per application, inside a moving time window — catches nearly every credential attack in its early stage.

Abnormal access is comparison. When an attacker uses stolen but valid credentials, there are no failures at all — the login succeeds. What gives them away is behaviour that does not match the account's history: an unfamiliar country, an unusual hour, a resource this person has never touched, a volume of downloads far above their normal. The system therefore learns a baseline for each identity and flags deviations from it.

4. What Exactly Gets Recorded — the four event families

Everything the control captures falls into four families. Each family has a defined list of events so that coverage can be checked rather than assumed.

4.1 Authentication events — proving who you are

Every attempt to establish identity is recorded, whether it succeeded or not.

- Interactive sign-in: username plus password, passkey, hardware security key or certificate.
- Single sign-on: the federated login, the assertion issued, and the application it was issued for.
- Multi-factor checks: which factor was used, and whether it was approved, denied, timed out or bypassed.
- Machine-to-machine authentication: API keys, client credentials, service account token requests.
- Token lifecycle: access token issued, refresh token used, token expired, token revoked.
- Session lifecycle: session created, extended, ended by the user, ended by timeout, ended by an administrator.
- Credential changes: password set or reset, MFA device enrolled or removed, recovery code generated or consumed.
- Every failure, with a precise reason code — wrong password, unknown account, account disabled, account locked, expired credential, MFA denied, MFA timeout, unsupported client.
- Context for each of the above: source IP address, resolved location, device identifier, client application and user agent.

What is deliberately never recorded: the password itself, the one-time code, the token value, or any private key. Where a token must be referenced, only a non-reversible identifier or hash is stored.

4.2 Authorization decisions — what you were allowed to do

Authentication ends at the front door. Authorization is every subsequent decision about which rooms you may enter, and each of those decisions is recorded.

- The allow decisions, which define the blast radius if that identity is ever compromised.
- The deny decisions, at exactly the same level of detail — a burst of denials from one identity is one of the clearest probing signals there is.
- The policy, role, group or permission that produced the decision, by identifier and version.
- The evaluation reason: which condition matched or failed — for example missing role, resource out of scope, outside permitted hours, or missing step-up authentication.
- The resource identity, type, owner, environment and data classification.

- Delegated and impersonated access, recording both the real actor and the identity being acted on behalf of.
- Time-bound or just-in-time elevations: when granted, by whom, for how long, and when they lapsed.

4.3 Certificate activity — the machine identity trail

Certificates are the identity cards services use to prove themselves to one another. A wrongly issued certificate is effectively a master key, and a silently expiring one is an outage, so the whole lifecycle is recorded.

- Certificate signing requests: who requested it, for which names (subject and subject alternative names), with which key algorithm and size.
- Approval steps, where issuance requires them, including the approver and the justification.
- Issuance: serial number, subject, alternative names, issuing authority, template or profile used, validity start and end.
- Renewal and re-key events, and whether the old certificate was retired.
- Revocation: who revoked it, the reason code, and confirmation that the revocation was published to the certificate revocation list and OCSP responder.
- Expiry warnings at defined thresholds, and the actual expiry event when it occurs.
- Changes to trust: adding or removing a certificate authority, editing a trust store, changing pinning configuration.
- Failed mutual-TLS handshakes with the reason — unknown authority, expired peer certificate, hostname mismatch, revoked certificate, unsupported cipher.

4.4 Privileged operations — the high-power actions

These are few in number but they are how a small compromise becomes a large one, so they are recorded with extra context.

- Administrative console, API and command-line actions in production environments.
- Configuration and infrastructure changes: firewall and network rules, identity policies, storage permissions, deployment settings.
- Secret and key usage: vault reads, credential checkout, signing key use, encryption key operations.
- Identity administration: creating or disabling accounts, granting or removing roles, adding members to privileged groups.
- Data movement at scale: bulk export, database dump, report download above a defined size or row count.
- Break-glass and emergency access, recording the justification, the ticket reference and the approver.
- Changes to the auditing capability itself: editing a detection rule, changing retention, disabling a log source, altering access to the audit store.

- For each: a summary of what changed from and to, in a form that omits secret values but preserves the meaning of the change.

5. The Audit Record — field by field

Every record shares a common core, then adds a block of fields specific to its family. The tables below list the fields, what each one means in plain terms, and an illustrative value.

5.1 Core fields — present on every single record

Field	Plain-English meaning	Example
event_id	A unique reference for this one record, so it can be cited exactly.	evt_9f3a41c7b2
event_time	When the action actually happened, always in UTC.	2026-09-01T10:15:04.221Z
received_time	When the pipeline received it. A large gap from event_time signals delay or clock drift.	2026-09-01T10:15:05.008Z
event_category	Which of the four families this belongs to.	authentication
action	What was attempted, from a fixed vocabulary.	authenticate / grant_role
outcome	How it ended.	success / failure / deny
reason_code	Why it ended that way, from a fixed list.	mfa_denied
trace_id	The shared reference tying together everything from the same original request.	4bf92f3577b34da6
schema_version	Which version of the record shape this is, so old records stay readable.	2.1

5.2 Actor and origin — the “who” and “from where”

Field	Plain-English meaning	Example
actor.id	Stable identifier that does not change if a name changes.	u_00417
actor.type	Whether this is a person, a service or the platform itself.	human / service / system
actor.display	Readable name, for humans reading the record.	jane.doe
actor.auth_method	How this actor proved identity for this session.	passkey / password+totp
actor.session_id	Which session the action belongs to.	sess_7c21ab
actor.roles	The roles held at the moment of the action.	[payments-reader]
actor.on_behalf_of	Present when someone acts for another identity.	u_00982
source.ip	Network address the request came from.	203.0.113.44

Field	Plain-English meaning	Example
source.geo	Country and region resolved from the address.	IN / Karnataka
source.device	Managed device identifier, where known.	dev_lap_2291
source.user_agent	Client software that made the request.	Chrome 141 / macOS

5.3 Resource and target — the “what to”

Field	Plain-English meaning	Example
resource.type	What kind of thing was acted upon.	database / document / role
resource.id	The identifier of that specific thing.	db_payments_prod
resource.owner	The team accountable for it.	team-payments
resource.classification	How sensitive it is; drives alert severity and retention.	restricted
resource.environment	Which environment it lives in.	production
target.id	For actions performed on another identity, who was affected.	u_00982
change.summary	A short before-and-after description, with secret values omitted.	roles: [reader] → [reader, admin]

5.4 Family-specific blocks

Family	Additional fields recorded
Authentication	auth.method, auth.factor_type, auth.factor_result, auth.token_ref (a hash, never the token), auth.session_action, auth.client_app.
Authorization	authz.policy_id, authz.policy_version, authz.decision, authz.decision_path (which condition decided it), authz.effective_role, authz.elevation_id where just-in-time access was used.
Certificate	cert.serial, cert.subject, cert.sans, cert.issuer, cert.template, cert.not_before, cert.not_after, cert.key_algorithm, cert.requester, cert.approver, cert.revocation_reason, cert.publication_status.
Privileged operation	priv.ticket_ref, priv.approver, priv.justification, priv.command_summary, priv.break_glass flag, priv.before_after.

5.5 Integrity and pipeline fields

Field	Plain-English meaning	Example
prev_hash	Hash of the previous record, forming the tamper-evident chain.	sha256:3ac9...
record_hash	Hash of this record's own content.	sha256:81be...
signature_ref	Which signed block this record belongs to.	blk_2026090110
producer.service	Which service emitted the record.	identity-api
producer.version	Which build emitted it, so bad deployments can be traced.	4.12.3
producer.host	Which host or container it came from.	pod-idapi-7f9c
sensitivity_tag	Drives who may read this record and for how long it is kept.	pii-masked

6. Components, End to End

The capability is built from twenty-eight components arranged in seven layers. Layers one to six are the path a record travels; layer seven applies to all of them. Each component below is described the same way: what it is in simple terms, exactly what it does, why it matters, and what would go wrong without it.

6.1 Layer 1 — Event Sources: where the evidence is produced

Nothing downstream can invent information that was never captured. These four components are the instrumentation built into the systems themselves, and they are the most important part of the design.

6.1.1 Authentication Service Instrumentation

In simple terms: The code inside the identity and login systems that writes down every attempt to prove who someone is.

What it does, in detail:

- Emits one record for every authentication attempt, successful or not, before any other processing continues.
- Records the method used — password, one-time code, push approval, passkey, hardware key or client certificate — and the result of each factor separately, so a failed second factor is distinguishable from a failed password.
- Records session and token lifecycle events: creation, refresh, expiry, explicit logout and administrative revocation.
- Records credential management: password resets, MFA enrolments and removals, recovery code use.
- Captures the origin: source address, resolved geography, device identifier where the device is managed, client application and user agent.
- Uses a fixed reason-code vocabulary for failures so that counting and grouping work reliably.
- Emits nothing secret — no password, no code, no token value; token references are one-way hashes only.

Why it matters: Authentication is where almost every attack begins, and the failure signal here is the cheapest, highest-yield detection available.

If this were missing: You could not distinguish a brute-force attack from a forgetful employee, and after an incident you could not prove who was inside the system.

6.1.2 Authorization / Policy Decision Instrumentation

In simple terms: The part of the system that answers “is this actor allowed to do this to this resource?” and writes down its answer.

What it does, in detail:

- Emits a record for every decision, capturing allows and denies at identical fidelity.

- Records the policy, role or permission identifier and version that produced the decision, so a later policy change can be correlated with a change in behaviour.
- Records the decision path — the specific condition that granted or refused access — rather than just the verdict.
- Records the resource identity, type, owner, environment and data classification, so severity can be judged automatically later.
- Records delegation and impersonation with both identities present.
- Records temporary elevations: who granted them, for how long, and whether they were used before lapsing.

Why it matters: Authentication tells you who came in; authorization tells you what they could reach. Only the pair together describes exposure.

If this were missing: An investigation would stop at the front door — you would know an account logged in but not whether sensitive data was reachable, let alone reached.

6.1.3 Certificate and PKI Instrumentation

In simple terms: Logging built into the machinery that issues, renews and revokes the digital certificates that services use to prove themselves.

What it does, in detail:

- Records every certificate signing request with the requester, the requested names and the key parameters.
- Records issuance in full: serial, subject, alternative names, issuer, template, validity window.
- Records renewals, re-keys and whether the superseded certificate was retired.
- Records revocations with reason code, and confirms publication to the revocation list and OCSP responder as a separate event.
- Emits expiry warnings at defined thresholds — typically 30, 14 and 7 days — and an event at actual expiry.
- Records changes to trust stores and certificate authorities, which are among the highest-impact changes possible.
- Records failed mutual-TLS handshakes at the terminating proxy, with the specific reason.

Why it matters: A wrongly issued certificate is a master key that looks entirely legitimate; certificate records are the only way to spot one being minted, and the only reliable defence against outages caused by silent expiry.

If this were missing: A rogue certificate would be indistinguishable from a valid one, and expiries would take down production services with no forewarning and no trail.

6.1.4 Privileged Operation Instrumentation

In simple terms: Logging around the small set of actions that change the system itself rather than merely use it.

What it does, in detail:

- Records administrative actions taken through consoles, APIs and command lines in production.
- Records infrastructure and configuration changes with a before-and-after summary.
- Records access to secrets and keys — which secret, by whom, for what purpose where a justification is required.
- Records identity administration: account creation and disabling, role grants and removals, privileged group membership changes.
- Records bulk data movement above defined thresholds, including exports, dumps and large report downloads.
- Records break-glass usage with justification, ticket reference and approver, and raises a notification at the moment it is used.
- Records changes to the auditing capability itself, treating them as the most sensitive category of all.

Why it matters: Privileged actions are rare but they are the mechanism by which a foothold becomes a full compromise.

If this were missing: An intruder who obtains administrative rights could quietly widen their access, and nothing would show it happening.

6.2 Layer 2 — Capture and Shaping: turning an action into a record

6.2.1 Audit Emitter SDK

In simple terms: A small shared library that every service imports, so that all services produce audit records in exactly the same way.

What it does, in detail:

- Exposes one simple call — record the actor, action, resource and outcome — and does everything else automatically.
- Fills in the repetitive fields itself: timestamp in UTC, service name, build version, environment, region, host and the current trace ID.
- Applies schema rules, redaction rules and sensitivity tagging centrally, so no team can accidentally opt out.
- Writes to the local buffer rather than the network, so a slow pipeline can never slow a user request.
- Publishes its own metrics: records emitted, records rejected by validation, records dropped through overflow.
- Is versioned and rolled out like any other dependency, so schema changes propagate in a controlled way.

Why it matters: One library is the only practical way to guarantee consistency across dozens of services written by different teams at different times.

If this were missing: You would end up with dozens of incompatible formats, and every detection rule would need to be written and maintained dozens of times over.

6.2.2 Canonical Event Schema

In simple terms: The agreed shape of a record — the fixed set of fields every event must carry, and the fixed vocabulary its values must come from.

What it does, in detail:

- Mandates the core fields: actor, resource, action, outcome, event time and trace ID.
- Defines optional blocks per family, so a certificate record can carry certificate details without forcing every other record to have empty fields.
- Constrains action names and outcome values to controlled lists, which is what makes counting and grouping reliable.
- Is explicitly versioned, with rules requiring that new versions add fields rather than repurpose existing ones.
- Is published as machine-readable definitions used both by the SDK and by the validation gate, so documentation and enforcement cannot drift apart.

Why it matters: Structure is what makes records analysable. The word “structured” in this project's title is doing real work.

If this were missing: Detection would collapse into fragile text matching that silently breaks the day a developer rewords a message.

6.2.3 Trace ID Generation and Propagation

In simple terms: One reference number, created when a request first enters the system and carried unchanged to everything that request touches.

What it does, in detail:

- Generated at the edge — gateway, load balancer or front door — if the incoming request does not already carry one.
- Passed in a standard header on every internal call, so services do not need bespoke agreements with each other.
- Stamped onto every audit record, application log line and metric produced while handling that request.
- Carried into asynchronous work: queue messages and scheduled jobs inherit the originating trace ID so the chain is not broken at a queue boundary.
- Accompanied by a span identifier so the order and nesting of steps within one trace can be reconstructed.
- Validated for format on arrival, and regenerated rather than trusted when it arrives from an untrusted external caller.

Why it matters: It converts a pile of unrelated records into a single readable story, which is the difference between a five-minute investigation and a five-day one.

If this were missing: Analysts would rebuild the sequence of events by hand from timestamps, slowly and with real risk of joining the wrong records together.

6.2.4 Validation Gate

In simple terms: A checkpoint that refuses to accept a record that is incomplete or malformed.

What it does, in detail:

- Rejects any record missing actor, resource, action, outcome, event time or trace ID.
- Rejects action names and outcome values that are not in the approved vocabulary.
- Rejects timestamps implausibly far from the present, which usually indicates clock drift or a replayed record.
- Sends rejected records to a quarantine stream together with the reason, rather than discarding them, so nothing disappears without a trace.
- Raises an alert when the rejection rate rises above a small baseline, since a spike almost always means a faulty deployment.

Why it matters: A partially filled record is more dangerous than no record, because it creates confidence that turns out to be unfounded.

If this were missing: Gaps would open silently and only be discovered during an incident, which is precisely the worst moment to discover them.

6.2.5 Redaction, Masking and Data Classification

In simple terms: The step that keeps sensitive content out of the audit trail while keeping the trail useful.

What it does, in detail:

- Blocks secrets outright: passwords, one-time codes, tokens, private keys, full payment card numbers.
- Masks or hashes personal data where policy requires, keeping a stable hash so the same person can still be correlated without storing the raw value.
- Replaces large payloads with a fingerprint — size and hash — instead of the content itself.
- Applies a sensitivity label to every record, which then drives who may read it and how long it is retained.
- Runs inside the shared library so it is applied uniformly, and is backed by scanning of the stream to catch anything that slips through.

Why it matters: Audit trails are read by more people than almost any other dataset; they must not become the place secrets leak.

If this were missing: The security control itself would become a security incident and a privacy breach.

6.3 Layer 3 — Transport: moving records without losing them

6.3.1 Local Durable Buffer

In simple terms: A small queue held on the local disk of each host or container, holding records briefly before they are shipped onward.

What it does, in detail:

- Accepts a write in a fraction of a millisecond so the user's request is never made to wait for logging.
- Lives on disk rather than in memory, so records survive a process crash or a restart.
- Keeps records in order and marks each as unsent until the downstream system confirms receipt.
- Is bounded in size, and on overflow drops in a controlled, counted and alerted way rather than consuming the host's disk and causing an outage.
- Reports depth and age of the oldest unsent record, which are the earliest indicators of a pipeline problem.

Why it matters: Networks and pipelines fail. Without a buffer, every hiccup becomes a permanent, unrecoverable hole in the evidence.

If this were missing: Any transient pipeline problem would silently erase the record of whatever happened during it.

6.3.2 Collector Agent

In simple terms: A lightweight process that reads the local buffer and delivers records to the central pipeline.

What it does, in detail:

- Batches and compresses records so that shipping is efficient at high volume.
- Authenticates itself with mutual TLS using its own certificate — which is issued and logged by the same PKI described earlier.
- Retries with exponential backoff when the pipeline is unavailable, and removes records from the buffer only after an explicit acknowledgement.
- Adds host-level context such as node, cluster and container identity.
- Reports its own lag, error rate and throughput so the pipeline can be monitored end to end.

Why it matters: Separating production of records from their delivery means a fault in one cannot break the other.

If this were missing: Application code would end up owning network retry logic, which is where reliability bugs concentrate.

6.3.3 Streaming Queue

In simple terms: The central highway all audit records travel along before they are stored and analysed.

What it does, in detail:

- Preserves order per source and gives every record a durable position, so consumers can resume exactly where they stopped.

- Absorbs sudden spikes — an incident produces a flood of events precisely when you least want to lose them.
- Allows several independent consumers to read the same stream: the archive writer, the SIEM, and any future analytics job.
- Retains a short replay window, typically days, so a consumer that fails can catch up without data loss.
- Is itself access-controlled and encrypted in transit and at rest.

Why it matters: It decouples producers from consumers, which is what allows new detections and new destinations to be added later without touching a single application.

If this were missing: Every new consumer would require changes to every producing service, so in practice new detections would never be added.

6.3.4 Enrichment and Normalization

In simple terms: The step that adds useful context and makes records from different sources directly comparable.

What it does, in detail:

- Resolves actor identifiers into department, manager, employment type and current employment status — a record showing action by a terminated employee should be immediately visible.
- Resolves resource identifiers into owner, environment, business criticality and data classification.
- Adds geolocation and network reputation for source addresses.
- Maps vendor-specific field names onto the canonical schema, so a firewall's field and an application's field become one searchable field.
- Marks records where enrichment failed, rather than silently omitting the context and creating a misleading gap.

Why it matters: It lets a rule express a business idea — “a privileged action on a production system holding customer data, by a contractor” — without joining five systems at query time.

If this were missing: Every alert would require manual lookups across several tools, which is exactly what makes response slow.

6.4 Layer 4 — The Append-Only Audit Store: the tamper-evident system of record

6.4.1 Append-Only (WORM) Storage

In simple terms: The place records finally land, configured so they can be added and read but never altered or removed.

What it does, in detail:

- Uses write-once, read-many features enforced by the storage platform itself, not by application code that could be bypassed.

- Grants no update or delete permission to any identity, including administrators and including the service that writes the records.
- Applies an immutable retention period to each object at the moment of writing, so the removal date is fixed at creation.
- Partitions data by date and source so that retrieval stays fast and retention rules are simple to apply.
- Encrypts at rest with keys managed separately from the storage administrators.

Why it matters: Append-only is the difference between a log and evidence. A record that can be edited proves nothing about anything.

If this were missing: The first action of a competent intruder — deleting the trace of their own activity — would succeed completely.

6.4.2 Hash Chaining and Signing

In simple terms: Cryptographic sealing that makes tampering detectable even if the storage platform itself is compromised.

What it does, in detail:

- Computes a hash of each record and stores the hash of the preceding record alongside it, forming a continuous chain.
- Groups records into blocks and signs each block's digest with a key held in a hardware security module or managed key service.
- Publishes the signed digest to a separate system under different administrative control, so no one person governs both the data and its proof.
- Runs a scheduled verifier that recomputes the chain and raises a high-severity incident on any mismatch or gap.
- Retains verification results as records in their own right, providing evidence that verification is actually performed.

Why it matters: Prevention can fail. Detection of tampering must not, because it is what allows you to state truthfully that the record is complete.

If this were missing: You could assert that the log was complete but never demonstrate it, which fails both audits and any serious legal test.

6.4.3 Segregated Access Control for the Audit Store

In simple terms: Deliberate separation between the people who are audited and the people who can read or administer the audit trail.

What it does, in detail:

- Grants read access to a small named group, ideally reporting through a different management line from the teams being audited.

- Grants delete rights to nobody at all; retention expiry is the only path by which a record ever leaves.
- Records every read of the audit store as an audit event of its own — the question “who has been looking at the logs?” must be answerable.
- Masks sensitive fields for standard roles and reveals them only to elevated roles, under approval.
- Requires approval for break-glass access to the store and raises a high-severity notification when it is used.

Why it matters: This is segregation of duties applied to evidence: the person capable of committing a wrongdoing must not be the person capable of concealing it.

If this were missing: The control would be theatre, since the party being audited would effectively hold its own evidence.

6.4.4 Retention, Archival and Legal Hold

In simple terms: The lifecycle policy for records: how long they are kept, where, and how they eventually leave.

What it does, in detail:

- Keeps a hot tier for recent months, optimised for fast search during live investigations.
- Moves older records to a cold archive that is far cheaper but still immutable and still verifiable.
- Sets retention by record type and sensitivity, always honouring whichever legal or regulatory requirement is longest.
- Supports a legal hold flag that suspends expiry for anything relevant to an investigation, dispute or regulatory request.
- Records the eventual deletion as an event in its own right, naming the policy that authorised it.
- Tests restoration from archive on a schedule, rather than assuming it will work when needed.

Why it matters: Too short and you cannot investigate a breach discovered months later; too long and you carry avoidable cost and privacy exposure.

If this were missing: Evidence would either vanish before it was needed, or personal data would be retained beyond what the law permits.

6.5 Layer 5 — SIEM and Detection: turning records into signals

6.5.1 SIEM Ingest and Parsing

In simple terms: The security analytics platform reading a copy of the stream and making it instantly searchable.

What it does, in detail:

- Consumes from the streaming queue independently, so a slow SIEM never delays the authoritative archive write.
- Parses every field into a search index while keeping the original record alongside for verification.
- Applies a data-model mapping so that a rule can be written once and apply across all sources.

- Tracks ingestion lag and per-source record counts against expected volumes, feeding the gap detection described later.
- Holds an explicitly non-authoritative working copy — for evidence, the append-only store is the reference.

Why it matters: The store exists for truth and retention; the SIEM exists for speed. Trying to make one system do both jobs does neither well.

If this were missing: Analysts would be querying cold archives, and detections would run hours or days behind events.

6.5.2 Repeated Authentication Failure Detection

In simple terms: Rules that count authentication failures across time and dimensions and fire when the count stops looking human.

What it does, in detail:

- Counts failures per account, per source address, per source network and per application, inside sliding time windows.
- Counts distinct accounts targeted from a single source, which is what reveals password spraying that stays under any per-account threshold.
- Watches for a success immediately following a burst of failures, the single most important pattern in the whole rule set.
- Applies specific rules to multi-factor patterns, such as repeated push denials followed by an approval.
- Treats repeated failures by service accounts as significant at very low thresholds, since automation does not mistype.
- Supports suppression for known-noisy sources, with a mandatory expiry date on every suppression so exceptions cannot become permanent.

Why it matters: It is the cheapest and highest-yield detection in security, and it catches attacks in their earliest and most reversible stage.

If this were missing: Slow, patient password guessing would succeed quietly, and the first sign of trouble would be the consequences.

6.5.3 Abnormal Access Detection

In simple terms: Baselines that learn what normal looks like for each identity, and rules that flag departures from it.

What it does, in detail:

- Builds a profile per identity: usual working hours, countries, devices, applications, resources touched and typical request volumes.
- Flags first-time access to a sensitive resource, especially when combined with any other signal.
- Flags geographic impossibility — two sessions from locations too far apart for the time between them.

- Flags dormant accounts becoming active again, and service accounts starting to behave like interactive users.
- Compares against a peer group: whether anyone else performing the same role ever does this at all.
- Watches for escalation chains, such as a role grant followed quickly by its first use and then a sensitive read.
- Decays old behaviour over time so profiles follow genuine changes in a person's job rather than freezing on day one.

Why it matters: When credentials are stolen but valid, there are no failures to count; only behaviour reveals that the wrong person is using them.

If this were missing: Insider misuse and session-hijacking attacks would be effectively invisible, because every individual action looks authorised.

6.5.4 Risk Scoring, Deduplication and Suppression

In simple terms: The layer that turns many small signals into a short, ranked list a human can actually work through.

What it does, in detail:

- Weights detections by context: a privileged action on production customer data outranks a failed login on a test system.
- Groups related detections about the same actor, resource or trace into a single case rather than many separate alerts.
- Removes duplicates and collapses repeats of the same underlying condition.
- Escalates severity when weak signals combine — failures, then a success, then a role grant, then an export.
- Requires every suppression rule to carry an owner and an expiry date, so tuning cannot quietly become blindness.

Why it matters: Alert fatigue is one of the most common reasons real breaches are missed; ranking is a security control in its own right.

If this were missing: The one alert that mattered would sit unread on page nine of a queue of near-identical noise.

6.6 Layer 6 — Alert, Response and Evidence: acting on what was found

6.6.1 Alert Routing and On-Call

In simple terms: Getting the right alert to the right person quickly, with enough context that they can act without hunting.

What it does, in detail:

- Routes by severity: informational to a chat channel, medium to a ticket queue, high to a paged responder.

- Includes the actor, resource, action, time window, counts and the trace ID directly in the alert body.
- Provides deep links straight into the relevant search and the matching runbook.
- Escalates automatically if an alert is not acknowledged within a defined time.
- Records acknowledgement and resolution times, which become the measurable service levels for the whole capability.

Why it matters: Detection without reliable delivery is just a dashboard that nobody happens to be looking at.

If this were missing: Alerts would fire into an empty room, and time-to-respond would depend on luck.

6.6.2 Analyst Triage and Trace-ID Pivot

In simple terms: The human investigation step, made fast by the correlation identifier designed in earlier.

What it does, in detail:

- Search by trace ID to replay one request end to end across every service that touched it.
- Search by actor across a time range to see everything that identity did, in order.
- Search by resource to see everyone who touched it and what they were allowed to do.
- Compare against the actor's baseline to judge whether behaviour is genuinely unusual or simply new.
- Record the verdict — true positive, benign, needs escalation — as a durable record attached to the case.

Why it matters: The explicit design goal is a five-minute answer to “what actually happened here?”

If this were missing: Investigations would take days and frequently end without a conclusion, which is corrosive to trust in the whole programme.

6.6.3 Automated Response and Containment

In simple terms: A small set of pre-approved actions the system may take by itself when the pattern is unambiguous.

What it does, in detail:

- Locks an account or forces a step-up challenge after a confirmed credential-attack pattern.
- Revokes active sessions and refresh tokens so a stolen session cannot simply continue.
- Revokes a compromised certificate and publishes the revocation immediately.
- Blocks or rate-limits an offending source address or network.
- Opens a ticket, snapshots the supporting evidence and notifies the resource owner automatically.
- Writes every one of its own actions as a privileged-operation audit record, citing the case that authorised it — the automation audits itself.

Why it matters: Attacks progress in minutes, and responders are not always awake; automated containment buys the time humans need.

If this were missing: Damage would continue to accumulate while an alert waited in a queue for business hours.

6.6.4 Dashboards, Reports and Compliance Evidence

In simple terms: The reporting surface that makes the audit trail useful outside of incidents.

What it does, in detail:

- Operational views: authentication success and failure rates, denial rates by service, certificates expiring in the next 30 days, privileged actions by team.
- Audit packs on demand: every privileged action on a given system for a given quarter, with approver and justification.
- Access reviews driven by real usage, so permissions that were never exercised can be safely removed.
- Coverage reporting: which services are emitting audit records, which are not, and which have regressed.
- Trend reporting on the capability itself — alert volumes, false-positive rates, time to acknowledge and time to resolve.

Why it matters: Most of the value of an audit trail is realised outside incidents, in reviews, evidence and everyday hygiene.

If this were missing: You would be passing audits by assertion rather than evidence, and permission sprawl would go unchecked.

6.7 Layer 7 — Cross-Cutting Controls: they apply at every stage

6.7.1 Trusted Time

In simple terms: Making certain that every clock in the estate agrees, so that “before” and “after” mean something.

What it does, in detail:

- Synchronises all hosts to trusted time sources, and monitors and alerts on drift.
- Stores event time in UTC in a standard format, alongside the time the pipeline received the record.
- Uses event time for detection windows, and flags records where the gap between event and receipt is unusually large.
- Treats an unsynchronised host as a finding, since its records cannot be safely ordered against others.

Why it matters: Time ordering underpins correlation, sliding-window detections and the evidential weight of the whole trail.

If this were missing: Sequences of events could not be established reliably, which undermines both investigation and any legal use.

6.7.2 Key and Secret Management

In simple terms: Care of the cryptographic material that the audit system itself depends on.

What it does, in detail:

- Generates and holds signing keys inside a hardware security module or managed key service; keys are never exported.
- Rotates keys on a schedule with overlapping validity, so historical records remain verifiable after rotation.
- Issues the collector agents' TLS certificates from the same PKI, and logs them like any other certificate.
- Separates keys used to sign audit data from keys used for anything else, limiting the impact of any single compromise.

Why it matters: If a signing key leaks, tamper-evidence disappears and every integrity claim becomes unprovable.

If this were missing: The integrity guarantees of the append-only store could not be defended under challenge.

6.7.3 Pipeline Health and Gap Detection

In simple terms: Auditing the auditor: continuous checking that records are actually still flowing.

What it does, in detail:

- Every source emits a synthetic heartbeat event on a fixed interval; a missing heartbeat pages someone.
- Detects volume anomalies — a source emitting far fewer records than usual is treated as an incident, not a quiet day.
- Monitors lag and backlog at the buffer, the queue and the SIEM.
- Alerts on rising schema-validation rejections, which almost always indicate a faulty release.
- Reports coverage: newly deployed services that are not yet emitting records are flagged automatically.

Why it matters: Silent failure is the normal failure mode of logging systems; the absence of records must be as loud as a bad record.

If this were missing: You would believe you were covered right up until the moment you needed the evidence and found none.

6.7.4 Testing and Assurance

In simple terms: Proving the whole capability works, rather than assuming it does.

What it does, in detail:

- Replays canned events in a test environment and asserts that the expected rules fire — detection rules are unit-tested like code.
- Runs periodic end-to-end tests that generate a real event and confirm it reaches the store and the SIEM within the target time.
- Verifies the hash chain on a schedule and reports the result as evidence.
- Performs restore tests from cold archive, confirming that old evidence is genuinely retrievable and readable.

- Includes an audit-readiness check in the release process, so no new service reaches production without emitting records.
- Uses purple-team exercises to confirm real attack techniques produce the alerts they should.

Why it matters: Controls that are never tested tend to be discovered broken at the worst possible moment.

If this were missing: Confidence in the capability would rest on hope, and the first real incident would be the first real test.

7. The Workflow, End to End

This section follows a single action from the moment it happens to the moment it becomes evidence or an alert. Every stage is described in the order it occurs.

Stage 1 — A request arrives and is given a reference number

- A request reaches the edge of the system: a person signing in, a service calling an API, or a scheduled job starting.
- The gateway checks whether the request already carries a trace identifier. If it does and the caller is trusted, it is kept; if it is absent or the caller is untrusted, a fresh one is generated.
- The trace ID is attached to the request context and will be passed automatically on every internal call made while handling it.
- No audit record is written yet, because nothing security-significant has happened. The reference number simply exists, ready to label everything that follows.

Stage 2 — Authentication happens and is recorded

- The caller presents a credential: a password, a passkey, a client certificate or a service token.
- The identity service verifies it and, where required, evaluates a second factor.
- One audit record is written regardless of the result: actor (or the identifier that was attempted, if the account does not exist), action = authenticate, resource = the application or realm, outcome, reason code, method, source address, device, and the trace ID.
- If authentication succeeds, a second record captures session or token creation, referencing the token only by a one-way hash.
- If it fails, the record feeds the counters that the failure-detection rules will read a moment later.

Stage 3 — Authorization is decided and recorded

- The service asks the policy engine whether this actor may perform this action on this resource.
- The engine evaluates roles, policies and conditions such as time of day, network location or the need for step-up authentication.
- An audit record captures the verdict and the reasoning: actor, action, resource with its type, owner and classification, the decision, the policy identifier and version, the specific condition that decided it, and the same trace ID.
- Denials are captured with identical detail to allows, because denials are where probing shows up.

Stage 4 — Certificate activity is recorded

- Where the request involved a service-to-service call, the mutual-TLS handshake either succeeds or fails, and any failure is recorded with its precise reason by the terminating proxy.

- Separately, certificate lifecycle events flow continuously: requests, approvals, issuance, renewal, revocation and expiry warnings.
- Lifecycle events triggered by a user request inherit that request's trace ID; scheduled activity is given its own job trace ID so it remains correlatable.

Stage 5 — A privileged operation is recorded

- If the request results in a high-power action — a role grant, a configuration change, a secret read, a bulk export — an additional record is written.
- It carries the target of the action, a before-and-after summary with secret values omitted, and where required the ticket reference, justification and approver.
- It carries the same trace ID, so the login, the permission check and the privileged action are all provably part of one story.

Stage 6 — The record is built, checked and cleaned

- The emitter library assembles the fields supplied by the service and adds the automatic ones: UTC timestamp, service name and version, environment, region, host, and the current trace and span identifiers.
- The validation gate confirms that all mandatory fields are present and that the action and outcome values come from the approved vocabulary. Anything failing is quarantined with a reason rather than silently dropped.
- Redaction removes secrets, masks personal data according to policy, and replaces oversized payloads with a fingerprint.
- A sensitivity label is attached, and the record is serialised with its schema version.

Stage 7 — The record is written locally

- The record is appended to the on-disk buffer on the local host and acknowledged immediately.
- The user's request continues without waiting for any network operation, so audit logging can never add user-visible latency or fail a business transaction.
- The record is marked unsent until the pipeline confirms receipt.

Stage 8 — The record is shipped

- The collector agent reads a batch from the buffer, compresses it, and opens an authenticated, encrypted connection to the central pipeline.
- On success it receives an acknowledgement and only then removes those records from the buffer.
- On failure it retries with increasing delay, while the buffer continues to accept new records; buffer depth and oldest-record age are published as metrics that will raise an alert if the outage persists.

Stage 9 — The record enters the central pipeline

- Records land in the streaming queue, are assigned a durable position and preserve their order within each source.
- The queue fans the same records out to independent consumers: the archive writer, the SIEM, and any additional analytics job — none of which can slow the others down.

Stage 10 — The record is enriched and normalised

- Actor identifiers are resolved into department, manager, employment type and current status.
- Resource identifiers are resolved into owner, environment, criticality and data classification.
- Source addresses gain geographic and network reputation context.
- Vendor-specific field names are mapped into the canonical schema so all sources become directly comparable.
- Any enrichment that fails is explicitly marked, so missing context is visible rather than misleading.

Stage 11 — The record is written to append-only storage

- The archive writer computes the hash of the record and links it to the hash of the previous record in that partition.
- The record is written to write-once storage with an immutable retention date applied at the moment of writing.
- Periodically the digest of a block of records is signed with a protected key, and the signature is published to a separate system under different administrative control.
- From this point the record is the authoritative system of record: readable by an approved few, alterable by nobody.

Stage 12 — The record is indexed for search

- A separate consumer delivers the same record to the SIEM, which parses every field into a search index while retaining the original alongside.
- This copy is explicitly a working copy for speed; where evidence is needed, it is drawn from the append-only store.

Stage 13 — Detection runs

- Counting rules update sliding windows for authentication failures across account, source address, source network and application dimensions.
- Baseline models compare the record against the actor's own history and against the behaviour of their peer group.
- Correlation rules look for sequences rather than single events, such as failures followed by a success followed by a privilege change.
- Any rule that matches produces a detection carrying a score and the specific records that triggered it.

Stage 14 — Detections become an alert

- Detections about the same actor, resource or trace are grouped into a single case, and their scores combined.
- Duplicates are collapsed and any active, unexpired suppression is applied.
- The resulting case is assigned a severity and routed accordingly — chat, ticket or page — carrying the actor, resource, counts, time window, trace ID, deep links and the relevant runbook.

Stage 15 — A human triages it

- The responder opens the case and pivots on the trace ID to replay the full request end to end.
- They pivot on the actor to see everything that identity did around the same time, and on the resource to see who else touched it.
- They compare the behaviour against the baseline to distinguish genuinely anomalous from merely new.
- They record a verdict, which becomes part of the evidence and also feeds tuning.

Stage 16 — Response and containment

- For clear-cut patterns, pre-approved automation has already acted: locking the account, revoking sessions and tokens, revoking a certificate or blocking a source.
- For everything else the responder executes the runbook, escalating to the resource owner or to incident management as appropriate.
- Every containment action, automated or manual, is written as a privileged-operation audit record citing the case that authorised it.

Stage 17 — Evidence, learning and retention

- The case, its evidence and its verdict are preserved alongside the underlying records.
- Thresholds are tuned, new detections are written, and any log source found to be missing or incomplete is fixed — this is the feedback loop shown in the architecture diagram.
- Records age from the hot tier into cold archive on schedule; a legal hold suspends expiry where an investigation or dispute requires it.
- At the end of retention, deletion occurs and is itself recorded, naming the policy that authorised it.

7.1 A worked example, start to finish

The following walks a realistic incident through the same seventeen stages, to show how the pieces fit together in practice.

- Over forty minutes, an account belonging to a contractor accumulates sixty failed sign-in attempts from an address in a country the contractor has never worked from. Each failure produces an authentication record with `outcome = failure` and `reason_code = bad_password`, all carrying their own trace IDs.

- The counting rule for failures per account crosses its threshold at ten failures in five minutes and raises a low-severity detection. Because the source address is also failing against eleven other accounts, the spraying rule raises a second detection.
- The sixty-first attempt succeeds. A record with outcome = success is written, and the success-after-failures rule immediately escalates the case to high severity.
- The abnormal-access baseline adds two more signals: the country is new for this identity, and the sign-in occurred well outside their normal working hours.
- Risk scoring merges all four detections into one case about one actor, ranks it high, and pages the on-call responder with the actor, the source address, the counts and the trace ID of the successful login.
- Automated containment, already triggered by the confirmed pattern, revokes the new session and forces a step-up challenge on the account.
- The responder searches the trace ID of the successful login and sees, in order: the authentication success, an authorization allow on an administrative console, a role grant adding the account to a privileged group, and the start of a bulk export.
- Because the role grant and the export are privileged operations, each has its own record naming the actor, the target, the before-and-after state and the same trace ID. The picture is complete within minutes.
- The responder disables the account, revokes the certificate the export tooling was using, and blocks the source network. Each of those three actions is itself written as a privileged-operation record citing the case.
- Afterwards, the case, every supporting record and the verdict are retained under legal hold. The spraying threshold is tuned, a new detection is written for role grants immediately followed by bulk exports, and one service found to be missing authorization records is added to the backlog.

8. Detection Logic in Detail

The project specifies two detection families explicitly: repeated authentication failures, and abnormal access. This section explains how each is actually built.

8.1 Repeated authentication failures

All of these rules are variations on one idea — count failures, then compare the count to what a human being plausibly produces. What differs is the dimension being counted and the window it is counted over.

Pattern	What it looks like in the records	Illustrative rule
Brute force on one account	Many failures against a single account, usually from one or a few sources, within a short period.	10 or more failures for one account within 5 minutes.
Password spraying	One source tries a small number of common passwords against very many accounts, staying under any per-account threshold.	20 or more distinct accounts failing from one source within 15 minutes.
Credential stuffing	High volume against one application, many accounts, many sources, with an unusually high failure ratio.	Failure ratio above 80% with more than 500 attempts in 10 minutes for one application.
Slow distributed guessing	Deliberately low rate per source, spread across many sources over hours or days.	50 or more failures for one account in 24 hours across 10 or more distinct sources.
Multi-factor fatigue	Repeated push notifications denied by the user, then eventually approved out of irritation.	5 or more denials in 10 minutes followed by an approval.
Success after failures	The dangerous case: a burst of failures ends in a successful sign-in.	Any success within 15 minutes of crossing any failure threshold. Always escalate.
Service account failures	Automation does not mistype, so any repetition is meaningful.	3 or more failures for a non-human identity in 10 minutes.
Lockout storm	Many accounts locked in a short window, often the visible symptom of spraying.	15 or more distinct accounts locked within 10 minutes.

How thresholds are chosen and kept honest

- Windows and thresholds are derived from measured data, not guessed. A baseline period establishes what normal failure rates look like per application and per population.
- Different populations get different thresholds. Customer-facing sign-in is noisier than internal administrative access and should not be held to the same numbers.

- The success-after-failure rule always fires, even when the failures alone stayed below every threshold. This deliberately catches the attacker who is patient enough to stay quiet.
- Known-noisy sources — an expired service credential, a misconfigured mail client — are suppressed only with a named owner and a mandatory expiry date on the suppression.
- Every rule is unit-tested with replayed events, so a change to log formats or thresholds cannot silently disable it.
- Rules record their own firing rate; a rule that has not fired in months is reviewed, because in practice it is often broken rather than lucky.

8.2 Abnormal access

These detections exist because a stolen but valid credential produces no failures at all. The sign-in succeeds; only the behaviour afterwards is wrong.

Signal	What it means in plain terms	Why it is suspicious
New geography	Access from a country or region this identity has never used.	Credentials are commonly used from wherever the attacker is.
Impossible travel	Two sessions from locations too far apart for the time between them.	One identity cannot be in two places; one of the two is not the owner.
Off-hours privileged action	An administrative action well outside this person's normal working pattern.	Attackers prefer times when nobody is watching.
First-time resource access	This identity has never touched this system or dataset before.	Reconnaissance and data collection look exactly like this.
Volume spike	Far more reads, downloads or exports than this identity normally performs.	Data theft is, mechanically, an unusually large read.
Dormant account waking	An account unused for a long period suddenly becomes active.	Forgotten accounts are attractive because nobody notices their use.
Peer-group outlier	Nobody else in the same role performs this action at all.	Suggests the permission is either misused or should not exist.
Escalation chain	A role grant, followed quickly by its first use, followed by a sensitive action.	This is the standard shape of an attacker expanding access.
Service account acting human	An automated identity signing in interactively or from a workstation.	Almost always means its credentials have been taken and reused.
Certificate anomaly	A certificate requested for a name outside the requester's normal scope, or with an unusually long validity.	A wrongly issued certificate is effectively a forged identity.

Signal	What it means in plain terms	Why it is suspicious
Denial spike from one actor	A single identity generating many authorization denials.	Classic probing for what a compromised account can reach.

How the baselines are built

- Each identity has a profile learned over an initial period, covering hours of activity, locations, devices, applications, resources and typical volumes.
- New identities with no history use their peer group's profile as a starting point, so there is no blind spot on day one.
- Profiles decay over time, so a genuine change of role updates the baseline instead of generating alerts forever.
- Known seasonality is accounted for, so month-end reporting or a planned migration does not look like data theft.
- A single weak signal produces a low-severity detection; high severity generally requires two or more corroborating signals, which is what keeps the alert volume workable.
- Every anomaly alert states plainly what was compared and what the normal value was, so a responder can judge it in seconds rather than trusting a score.

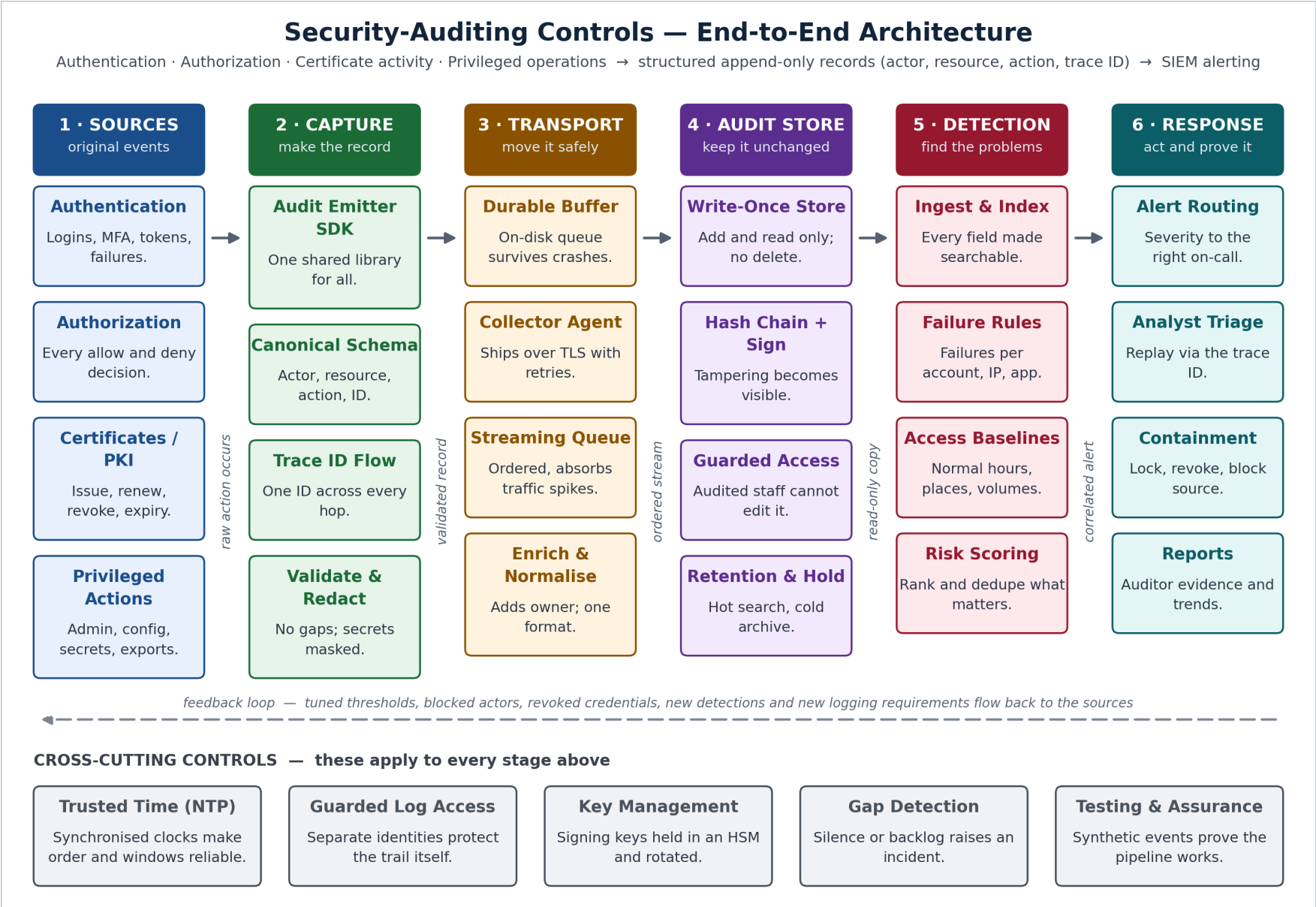
9. Architecture Diagram and Its Explanation

The diagram on the following page shows the whole capability on one sheet. It reads left to right in six numbered stages, following the journey of a single audit record from the moment an action occurs to the moment it becomes an alert or a piece of evidence.

How to read it

- The six coloured columns are the six stages, in order. Each column header names the stage and, underneath, states its job in three or four words.
- The boxes inside each column are the individual components of that stage, described in detail in Section 6.
- The arrows between the columns show the direction of flow, and the small italic label in each gap names what is being handed from one stage to the next.
- The dashed arrow running back along the bottom is the feedback loop: what is learned at the response stage changes the rules, the credentials and even the logging requirements back at the source.
- The grey strip at the very bottom holds the cross-cutting controls. They are drawn separately because they are not a stage — they apply to every stage above them at the same time.

Figure 1 — Security-auditing controls: end-to-end architecture



9.1 Walking the diagram, column by column

Column 1 — Sources: the original events

This is where evidence is created, and nothing downstream can recover what is not captured here.

- **Authentication** — The login systems record every identity check and every failure.
- **Authorization** — The policy engine records every allow and every deny, with the policy that decided it.
- **Certificates / PKI** — The certificate authority records issuance, renewal, revocation and expiry.
- **Privileged Actions** — Administrative surfaces record configuration changes, secret access, role grants and exports.

Column 2 — Capture: making the record

A raw action is turned into a consistent, complete, safe record before it goes anywhere.

- **Audit Emitter SDK** — The single shared library used by every service, so every record has the same shape.
- **Canonical Schema** — The fixed field set: actor, resource, action, outcome, time and trace ID.
- **Trace ID Flow** — The shared reference number carried across every hop of the same request.
- **Validate & Redact** — Incomplete records are rejected and quarantined; secrets and personal data are removed or masked.

Column 3 — Transport: moving it safely

The record travels from the machine that produced it to the systems that store and analyse it, without being lost or delaying anything.

- **Durable Buffer** — An on-disk queue on the local host so records survive crashes and outages.
- **Collector Agent** — Ships batches over authenticated, encrypted connections and retries until acknowledged.
- **Streaming Queue** — The ordered central pipeline that absorbs spikes and feeds several consumers at once.
- **Enrich & Normalise** — Adds owner, classification and location context, and maps every source into one common format.

Column 4 — Audit Store: keeping it unchanged

This is the authoritative record, and its defining property is that nobody can alter it.

- **Write-Once Store** — Records can be added and read but never edited or deleted; enforced by the storage platform.
- **Hash Chain + Sign** — Each record seals the previous one, and blocks are signed, so tampering is detectable.
- **Guarded Access** — The people being audited cannot administer the trail, and every read is itself recorded.

- **Retention & Hold** — A fast hot tier, a cheap immutable archive, and legal holds that freeze evidence when required.

Column 5 — Detection: finding the problems

The SIEM reads a working copy of the same records and continuously looks for trouble.

- **Ingest & Index** — Parses every field into a fast search index for both rules and analysts.
- **Failure Rules** — Counts authentication failures per account, per source and per application inside moving windows.
- **Access Baselines** — Learns each identity's normal hours, places, devices and volumes, and flags departures.
- **Risk Scoring** — Combines and ranks signals, removes duplicates, and decides what deserves a human.

Column 6 — Response: acting and proving

Findings become actions, and actions become evidence.

- **Alert Routing** — Delivers the right severity to the right on-call channel with the evidence attached.
- **Analyst Triage** — The responder pivots on the trace ID to replay the entire request end to end.
- **Containment** — Locks accounts, revokes sessions and certificates, blocks sources — and audits itself doing so.
- **Reports** — Dashboards, audit packs, access reviews and coverage reporting.

The labels between the columns

Each gap in the diagram is labelled with what is being passed forward, because the hand-off is where most designs go wrong.

- **“raw action occurs”** — Between Sources and Capture. Something real has happened, but it is not yet a record.
- **“validated record”** — Between Capture and Transport. It is now complete, well-formed and safe to move.
- **“ordered stream”** — Between Transport and Audit Store. Records arrive in order, enriched, ready to be sealed.
- **“read-only copy”** — Between Audit Store and Detection. The SIEM receives a copy; the authoritative record stays untouched.
- **“correlated alert”** — Between Detection and Response. Many signals have become one ranked case with evidence attached.

The feedback loop

The dashed arrow returning from Response to Sources is a genuine part of the architecture, not decoration. What is learned at the right-hand side changes the left-hand side: thresholds are tuned, credentials are revoked, attacking sources are blocked, new detections are written, and — most importantly — any missing or

incomplete log source discovered during an investigation becomes a requirement back at the instrumentation layer. A capability without this loop degrades quietly over time.

The cross-cutting strip

The five grey boxes along the bottom apply to every column simultaneously, which is why they are drawn outside the flow.

- **Trusted Time (NTP)** — Synchronised clocks, so ordering and time-window rules can be relied upon.
- **Guarded Log Access** — Separate identities and approvals protecting the audit trail itself.
- **Key Management** — Signing and TLS material held in protected hardware and rotated on schedule.
- **Gap Detection** — Heartbeats and volume monitoring, so silence raises an incident instead of going unnoticed.
- **Testing & Assurance** — Synthetic events, rule tests and integrity verification proving the pipeline actually works.

10. Design Rules and Non-Functional Requirements

These are the standing rules the implementation must obey. They are the constraints that keep the capability trustworthy under load and over time.

- **Auditing must never break the business.** Emitting a record is asynchronous and locally buffered. A failure anywhere in the pipeline must never fail, delay or block the user's transaction.
- **Speed targets.** Local emission in well under a millisecond; end-to-end delivery to the SIEM within seconds under normal load and within a few minutes under incident-level load.
- **Completeness beats convenience.** It is better to raise an alert about a missing record than to lose one quietly. Absence of data is treated as a finding.
- **Everything in UTC.** One time zone, one format, everywhere, with the pipeline's receipt time recorded alongside the event time.
- **No secrets, ever.** Passwords, codes, tokens and private keys never enter the trail in any form, in any environment.
- **No delete permission anywhere.** Retention expiry is the only mechanism by which a record leaves the store, and that expiry is itself recorded.
- **The audit path is production.** It has owners, service-level objectives, capacity planning, on-call cover and change control, exactly like a customer-facing system.
- **Size for the worst day.** Capacity is planned around incident-day peaks, when volumes multiply and evidence matters most, not around average load.
- **Never sample audit events.** Sampling is acceptable for general telemetry. It is never acceptable for audit records, because the missing one is always the interesting one.
- **Cost is managed by tiering, not by dropping.** Hot storage for recent records, cheap immutable archive for the rest.
- **No production without instrumentation.** Every new service passes an audit-readiness check before release, and coverage reporting flags any regression.
- **Every exception expires.** Suppressions, tuning exceptions and elevated access all carry a named owner and a mandatory expiry date.

11. Failure Modes and Safeguards

Every logging system fails in broadly the same ways. These are the ones this design anticipates explicitly.

Failure mode	What it looks like	Safeguard built into the design
Logging slows the application	User-visible latency spikes that correlate with pipeline load.	Emission is asynchronous into a local on-disk buffer; the request never waits for the network.
Pipeline outage	A gap in the record for the duration of the outage.	On-disk buffering, acknowledged delivery, retry with backoff, and alerting on buffer depth and age.
A source stops sending	Dashboards go unusually quiet and nobody notices.	Synthetic heartbeats per source plus volume-anomaly detection; missing records page someone.
A developer rewords a message	Detection rules quietly stop matching.	Controlled vocabularies, schema validation and unit-tested rules that fail the build if they stop firing.
Clock drift	Events appear out of order; time-window rules under-count.	Monitored time synchronisation, plus alerting when event time and receipt time diverge.
A secret is written to a log	A new incident created by the security control itself.	Central redaction in the shared library, plus continuous scanning of the stream for secret patterns.
Too many alerts	The one that mattered is lost in the noise.	Risk scoring, grouping into cases, deduplication and suppressions that must carry an expiry date.
An intruder tries to erase evidence	Records missing around the time of the intrusion.	Write-once storage with no delete permission, hash chaining, and signed digests held elsewhere.
An alert fires and nobody acts	Long dwell time between detection and response.	Severity-based routing, automatic escalation on non-acknowledgement, and measured response times.
Archive cannot be read when needed	An investigation stalls on unreadable or unrestorable evidence.	Scheduled restore tests and periodic hash-chain verification, with results retained as evidence.
Schema change breaks old records	Historical searches return nothing.	Versioned schema with additive-only changes, and version-aware parsing in the SIEM.

Failure mode	What it looks like	Safeguard built into the design
Suppression becomes permanent blindness	A whole class of events silently ignored for years.	Every suppression has a named owner and a mandatory expiry, and is reviewed on expiry.

12. Glossary

Term	Plain-English meaning
Actor	The identity that performed an action — a person, a service account or the platform itself.
Resource	The thing that was acted upon: an account, a document, a database, a certificate, a configuration item.
Action	What was attempted, taken from a fixed list such as authenticate, read, export or grant_role.
Outcome	How the action ended: success, failure or deny, with a reason code from a fixed list.
Trace ID	A reference number created when a request first arrives and carried through every service it touches, so related records can be found together.
Append-only	Records can be added and read but never changed or removed.
WORM	Write Once, Read Many — storage that physically enforces append-only behaviour.
Hash chain	Each record stores a fingerprint of the previous one, so altering any record breaks every link after it.
SIEM	Security Information and Event Management — the platform that indexes security records, runs detection rules and raises alerts.
Baseline	A learned picture of normal behaviour for one identity or service, used to judge whether new behaviour is unusual.
PKI	Public Key Infrastructure — the system that issues and manages the digital certificates services use to prove their identity.
mTLS	Mutual TLS — a connection where both sides prove their identity with certificates, not just the server.
CRL / OCSP	The two mechanisms by which a revoked certificate is published as no longer trustworthy.
HSM / KMS	Hardware Security Module or Key Management Service — protected storage for cryptographic keys, from which keys cannot be exported.
Segregation of duties	Deliberately splitting powers so the person who could do something wrong is not the person who could conceal it.
Legal hold	A flag that suspends normal deletion because the records are relevant to an investigation or dispute.
Privileged operation	A high-power action that changes the system itself rather than merely using it.
False positive	An alert that turns out to describe legitimate activity.

Term	Plain-English meaning
Dwell time	How long an attacker is present before being detected — the number this whole capability exists to reduce.
Password spraying	Trying a few common passwords against very many accounts, to stay below per-account lockout thresholds.
Credential stuffing	Replaying username and password pairs stolen from elsewhere against your systems at high volume.
Break-glass	Emergency access granted outside normal process, always justified, always recorded and always reviewed.